Below is an explanation and a complete JavaScript solution using a merge sort approach for the Big Sorting problem. In this problem, the array

1. **By Length:** Since the numbers are positive and contain no leading zeros, a string with fewer digits represents a smaller number. For exampl
2. **Lexicographical Comparison:** If two strings are of the same length, then comparing them lexicographically (i.e. dictionary order) will corre

The merge sort approach recursively splits the array into halves and then merges them back together using a custom comparator that impleme

Below is the code solution:

Js Copy

```
function bigSorting(unsorted) {
  // Use our merge sort function to sort the array.
  return mergeSort(unsorted);
}

function mergeSort(arr) {
  // Base case: arrays with 0 or 1 element are already sorted.
  if (arr.length <= 1) return arr;

  // Find the middle index.
  const mid = Math.floor(arr.length / 2);

  // Recursively sort the left and right halves.
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));

  // Merge the sorted halves.
  return merge(left, right);
}

function merge(left, right) {
  const sorted = [];
  let i = 0, j = 0;

  // While both arrays have elements, compare the current elements using our custom comparator.
  while (i < left.length && j < right.length) {
    if (compareNumericStrings(left[i], right[j]) <= 0) {
      sorted.push(left[i]);
      i++;
    } else {
      sorted.push(right[j]);
      j++;
    }
  }

  // If any elements remain in the left or right, add them to the result.
  while (i < left.length) {
    sorted.push(left[i]);
    i++;
  }
  while (j < right.length) {
    sorted.push(right[j]);
    j++;
  }

  return sorted;
}

function compareNumericStrings(a, b) {
  // Compare by length first.
  if (a.length !== b.length) {
    return a.length - b.length;
  }
  // If lengths are equal, compare lexicographically.
  return a.localeCompare(b);
}

// Example usage:
// Suppose we have an unsorted array of numeric strings:
const unsorted = [
  "31415926535897932384626433832795",
  "1",
  "3",
  "10",
  "3",
  "5"
```

```
];

// The solution returns the sorted array:
const sorted = bigSorting(unsorted);
console.log(sorted);
```

Explanation Summary

- **bigSorting** calls **mergeSort** on the unsorted array.
- **mergeSort** recursively divides the array until every piece has one or zero elements.
- **merge** combines two sorted arrays into one. During merging, the custom function **compareNumericStrings** is used:
  - It first compares the lengths to determine the numerical order.
  - If the lengths are equal, it compares the strings lexicographically.
- The final output is the array sorted in ascending order based on their numerical value.

This approach avoids converting large numbers into actual numeric types (which could lead to precision issues) and directly compares the strin